Geavanceerde Computerarchitectuur — Session 1: teaching notes

Audience: 3rd-year electronics students. They know C/C++, digital electronics and microcontrollers. Most have never programmed a GPU.

Slide 3 — GPUs can execute more threads simultaneously

Goal. Students understand why a GPU exists: it gives up single-thread speed to gain throughput.

Present

- **Start from the figure.** One process has two pools of threads.
  - Point to the large yellow (control) and purple (cache) areas on the CPU side, then to the thin strips on the GPU side.
  - The message: the CPU spends its transistors on making a few threads fast, and the GPU spends them on arithmetic units.
- **Name the trade-off: latency versus throughput.** Analogy: a sports car (CPU) gets one person somewhere fast. A bus (GPU) is slower per trip but moves fifty people at once.
- **Explain latency hiding.**
  - A memory access takes hundreds of clock cycles.
  - A CPU hides that delay with large caches and speculation.
  - A GPU keeps thousands of threads resident. When one group waits for memory, the scheduler runs another group in the next cycle. The switch costs nothing, because every thread's registers stay in the register file.
- **Fix the vocabulary early.**
  - A "CUDA core" is one arithmetic lane, not a core in the CPU sense.
  - The GPU equivalent of a CPU core is the Streaming Multiprocessor (SM).
  - Threads run in groups of 32 (warps). The details come in the Warps session.
- **Give one number for scale.** An RTX 4090 has 128 SMs, 16,384 lanes and about 196,000 resident threads. A desktop CPU has 16–32 hardware threads. Use the lab GPU's numbers if you have them.
- **Electronics angle.** Clock frequency stopped scaling because of power. Since then, performance comes from more parallel arithmetic per watt.

Ask the class: "Why can a GPU get away with so little cache?" Answer: enough threads in flight hide the memory delay.

Expected questions

- *Why not run everything on the GPU?* Serial code, branch-heavy code and small problems run badly on a GPU, and data transfer over PCIe costs time. Amdahl's law: if 90% of the program is parallel, the maximum speedup is 10×, however many cores you add.
- *Is a GPU core slower than a CPU core?* Per thread, yes. In total throughput the GPU is far faster.

Prepare: the lab GPU's SM count, maximum threads per SM and compute capability.

Slide 4 — Example: parallel processing for ray tracing

Goal. Give a concrete workload that fits slide 3, and introduce the idea of independent work items.

Present

- **Explain how the image is made.**
  - For every pixel, a ray leaves the camera and the nearest sphere it hits is found.

  - The ray then bounces: it scatters off diffuse surfaces, reflects off metal and refracts through glass. This repeats until the ray escapes or a depth limit is reached.
  - Many samples per pixel are averaged.
- **The key point.** No pixel depends on another pixel, so each pixel can be one thread. Full HD with 100 samples per pixel is about 200 million independent rays. The term for this is *embarrassingly parallel*.
- **Foreshadow.** The image is 2D, so the threads will be arranged in 2D blocks (slides 12–18).
- **Be honest about the limits.**
  - Neighbouring rays that hit different materials follow different paths. This is called divergence and comes back in the Warps session.
  - Each thread needs its own random numbers.
  - Modern GPUs have dedicated RT cores for ray tracing. This example uses only the general-purpose cores.
- **Electronics examples with the same structure:**
  - FIR filters and FFTs per channel
  - beamforming per angle
  - radio propagation ray tracing
  - Monte Carlo simulation
  - image processing
  - AES in counter mode

  Several of these return as project topics (slide 8).

Ask the class: "Which of these can run in parallel: vector addition, running sum, IIR filter, histogram, matrix multiplication?"

- Vector addition and matrix multiplication: yes.
- Histogram: yes, but threads collide on the same bins. That is the topic of session 2.
- Running sum and IIR filter: not in their naive form, because every output depends on the previous one.

Expected question: *Why one thread per pixel and not one per bounce?* The bounces of one ray are sequential: each depends on the previous hit.

Slide 5 — Relationship between architecture and scope

Goal. Students know which part of a CUDA program runs where, and that the CPU stays in control.

Present, overlay by overlay

1. **Base.** One source file in five parts. Everything is compiled and started on the CPU.
2. **Arrows to the GPU.** Kernel and device functions run on the GPU. Parts 3–5 are the interface: they run on the CPU but drive the GPU.
3. **Call chain.** The launch in part 4 starts the kernel, and the kernel calls the device functions.

Explain in words

- **Three function markers.**
  - A *kernel* (`__global__`) is started by the CPU, runs on the GPU and returns nothing. Starting it does not wait for it to finish.
  - A *device function* (`__device__`) is a helper that only GPU code can call.
  - Unmarked functions are ordinary CPU functions. A function marked for both host and device is compiled twice.
- **Compilation.** `nvcc` splits the file. The CPU part goes to the normal C++ compiler, and the GPU part is compiled to GPU machine code. Both end up in one executable.
- **Memory.** A discrete GPU has its own memory, and there are two ways to deal with that:
  - **Unified Memory:** one pointer that is valid on both sides; the driver moves ("migrates") the data when it is needed. This is the "migration" on the slide.

  - **Explicit:** allocate on the GPU and copy the data across in each direction yourself.
- **Asynchronous launch.** The CPU continues immediately after starting a kernel. Before reading results or stopping a timer, the program must wait for the GPU. For timing, use CUDA events, not a CPU timer.
- **Errors are silent.** They appear only if you check for them, after the launch and after waiting. Always compare the result against a CPU reference.
- **Analogy for electronics students.** A microcontroller with a DMA peripheral: set up the buffers, start the transfer, wait for the done flag or interrupt, then read the buffer.

Ask the class: "What happens if you read the results on the very next line after the launch?"

Expected questions

- *Can a kernel return a value?* No. Results are written to memory through pointers.
- *Can a kernel print?* Yes. The output is buffered and appears when the CPU waits for the GPU.
- *Can the CPU call a device function?* No, that is a compile error.

Prepare: check how the course's `tasks.json` compiles. The slide names the file `code.cpp`, but `nvcc` only treats a `.cpp` file as CUDA when it is given the `-x cu` option.

Slide 6 — CUDA, an API for programming Nvidia GPUs

Goal. Present CUDA as a full platform and introduce compute capability.

Present

- **Walk the stack from top to bottom:** applications; libraries (cuBLAS, cuFFT, cuDNN, Thrust, ...); languages (C/C++, Fortran, Python wrappers, OpenACC); the hardware.
- **CUDA is more than a language extension.** It consists of:
  - a programming model;
  - the `nvcc` compiler;
  - the runtime API, which is what this course uses (a lower-level driver API also exists but is not needed here);
  - libraries;
  - tools such as compute-sanitizer, Nsight and cuda-gdb.
- **Practical rule.** If a library already does it, use the library. In this course we write our own kernels to learn how things work underneath.
- **Compute capability.** This is the version number of a hardware generation. It sets the limits, for example the maximum number of threads per block. The tables are at the end of the Programming Guide. Tell the students the compute capability of the lab GPU.
- **Update the figure verbally.**
  - It stops at Ampere. Newer generations: Ada (8.9), Hopper (9.0), Blackwell (10.x and 12.x).
  - CUDA 13 dropped support for Maxwell, Pascal and Volta.
  - The brand names Tesla and Quadro are no longer used.
- **Portability.** CUDA runs only on NVIDIA GPUs.
  - AMD's HIP is nearly identical; OpenCL and SYCL are the cross-vendor options.
  - The concepts transfer directly: a warp on NVIDIA is a wavefront on AMD.

Expected questions

- *Is CUDA a language?* It is C++ with a few extensions, plus a runtime library.
- *Can I use it from Python?* Yes: CuPy, Numba, PyTorch.
- *Does a compiled program run on another GPU?* Usually yes. The driver can recompile the embedded intermediate code for newer GPUs.

Slide 7 — Environment and workflow

Goal. Students can build and run a program.

Present

- **The path.**
  1. Write C++ in VS Code.
  2. `tasks.json` holds the compiler command and its options.
  3. The executable runs on the CPU, which uses the GPU.
  4. Output goes to the terminal or to a file.
- **The GPU has no input or output of its own.** All output passes through the CPU. Printing from a kernel is buffered, and the output is lost if the program ends without waiting for the GPU.
- **Large results.** Dump them to a file and check them offline, for example in Python.
- **"Virus (…)".** The slide doesn't explain it. Decide how you want to present it, or drop it.

Prepare: be able to explain every option in the course's `tasks.json`.

Slide 8 — Organization of the lab and docs

Goal. Students see the arc of the whole course.

Present

- **Docs on Toledo:** the programming guide, the cheat sheet and `tasks.json`.
- **The six sessions, in one sentence each:**
  - **Kernel:** launching, indexing and memory migration (today).
  - **Race conditions:** threads writing to the same address; atomic operations, synchronization within a block, reductions.
  - **Warps:** 32 threads in lockstep, and what happens when they branch differently.
  - **Memory:** the memory types, access patterns, and shared-memory bank conflicts.
  - **Async:** streams, and overlapping data transfer with computation.
  - **Dynamic parallelism:** kernels that launch other kernels.
- **Project topics** (encoding, encryption, DSP, graphics) are all workloads of the kind shown on slide 4.

Prepare: since CUDA 12, a kernel can no longer wait for the kernels it launched itself. Check the old material for the Dynamic parallelism session against this change.

Slide 9 — Responsibilities and evaluation

Present

- **The weighting.**
  - Lab sessions count for 50%: six single-page reports, one per session.
  - The project counts for 50%: a report, a presentation and a code release.
- **What a good single-page report contains:**
  1. The problem, in one line.
  2. The chosen launch configuration and indexing, with the reason.
  3. How correctness was verified.
  4. Timing results and the method used, where relevant.
  5. Answers to the open questions on the slides.

Prepare: the grading rubric, from the course owner.

Slide 10 — Recap

Use the slide as a check. Ask three students in turn:

- the first to name the five parts of a program and say where each runs;
- the second to describe the build path;
- the third to list the six sessions.

Slide 12 — Threads are organized into a grid

Goal. Students understand the hierarchy grid → blocks → threads, and how the launch configuration creates it.

Present

- **The launch gives the pool its structure.** The two numbers between the triple angle brackets are the number of blocks and the number of threads per block. In the example, 2 blocks of 2 threads give 4 threads.
- **Total threads** = number of blocks × threads per block. Both can be 1D, 2D or 3D.
- **Why blocks exist.**
  - A block runs entirely on one SM, so its threads can share fast memory and wait for each other.
  - Blocks are independent: they can run in any order, on any SM. That is why the same program scales from a small GPU to a large one.
  - The consequence: blocks cannot wait for each other inside a kernel.
- **Limits.** At most 1024 threads per block. The grid can be very large (up to $2^{31}-1$ blocks in x).
- **Warps.** The hardware splits every block into warps of 32, so block sizes should be multiples of 32. A block of 2 threads leaves 30 lanes idle.

Ask the class:

- "Why not one big block?" Answer: it runs on one SM only, and it is capped at 1024 threads.
- "Why not blocks of 1 thread?" Answer: 31 of the 32 lanes are wasted.

Demo idea. Launch a kernel that prints its block and thread number, first with 2 × 2 and then with 4 × 8. Show that the order of the blocks changes from run to run.

Slide 13 — Calling a kernel function: example

Goal. Pose the two questions that every kernel launch has to answer.

Present

- **Goal on the slide:** one thread per input element, for 12 elements. This is not always the mapping: later we see threads that handle several elements, and several threads that work on one element.
- **OQ1:** how do we choose the block and grid size?
- **OQ2:** how does a thread know which element is its own?

Ask the class to answer both before you move on, and collect the answers on the board.

Slide 14 — OQ1: You decide how to divide the grid

Present

- **Both work.** 1 block of 12 threads and 3 blocks of 4 threads both give 12 threads, and the programmer chooses.
- **Consequences.**
  - One block of 12 uses a single SM and can never exceed 1024 elements.
  - Three blocks of 4 use three SMs, but each block of 4 threads occupies a full warp of 32 lanes.
- **Rule of thumb.**
  1. Fix the block size at a multiple of 32, typically 256 (128–512).
  2. Number of blocks = N divided by the block size, rounded up.
  3. Rounding up creates surplus threads, which need a bounds check (slide 17).
- **Work an example on the board.** N = 1000 with block size 256 gives 4 blocks, so 1024 threads, of which 24 are surplus.

Expected question: *Which split is fastest?* It depends on the kernel, so measure it. The Warps and Memory sessions explain why.

Slides 15–16 — OQ2: Importance of correct indexing

Present

- **One block (1 × 12) works.** The thread's number within the block identifies the element.
- **Three blocks (3 × 4) break.** If a thread uses only its position within the block, thread 0 of every block handles element 0, thread 1 handles element 1, and so on. Elements 0–3 are processed three times, and elements 4–11 are never processed.
- **Two failures result.**
  - **Overwriting.** If the same value is simply written three times, the result is only wasteful. For read-modify-write operations (such as incrementing) or in-place operations (such as the flip), the results become wrong and differ from run to run. That is a race condition, the topic of session 2.
  - **Skipping.** Two thirds of the output is never computed and contains whatever was in memory before.
- **Stress this:** the bug is invisible while everything fits in one block. It appears as soon as N exceeds 1024 or the block size changes, so always test with several blocks.

Ask the class: "If every thread adds 1 to its element, what does a zero array look like after a 3 × 4 launch?" Answer: usually 3, 3, 3, 3 followed by eight zeros, but that is not guaranteed.

Slide 17 — OQ2: Correct indexing

Present

- **The thread's position within its block is relative.** Add an offset of block index × block size to get the absolute position.
- **Draw it:** block 0 handles elements 0–3, block 1 handles 4–7, block 2 handles 8–11.
- **The formula to remember:** global index = block index × block size + thread index within the block.
- **The bounds check.** Because the number of blocks is rounded up, threads whose index is N or larger must do nothing. Otherwise they write outside the array. That may not crash, but it silently corrupts memory.
- **2D.** Apply the same formula per axis. The position in a row-major array is y × width + x.

- **Grid-stride loop** (mention briefly). Each thread processes its own element, jumps ahead by the total number of threads in the grid, and repeats until the end of the array. A fixed grid can then handle any N.

Slide 18 — Useful built-in functions

Present

- **Four built-in values** are available inside a kernel: thread index, block index, block size and grid size (`threadIdx`, `blockIdx`, `blockDim`, `gridDim`). Each has an x, y and z component. There is also the warp size, which is 32.
- **Correct the slide:** these are built-in variables, not functions.
- **Answer to the slide's question:** 1024 threads per block. x and y can each be up to 1024, z up to 64, and their product at most 1024.
- **Exceeding the limit.** A launch with more threads per block fails silently, unless the program checks for errors; the error message is "invalid configuration argument".
- **Total threads in the grid** = grid size × block size.

Prepare: check which document "reference guide p. 12" refers to, and whether the page number is still correct.

Slide 20 — First session's exercise

Present

- **The problem:** flip an array, using multiple blocks of multiple threads.
- **What students must show:** they migrate the memory and verify the result.
- **The report** is due by the next lab session.

Tips to give

- Use a separate output array.
- Test with an N that is not a multiple of the block size, and with N larger than 1024.
- Verify the result on the CPU.

Do not give away the index mapping. Hint instead: "If thread i reads element i, where must it write?"

Mistakes to watch for during the lab

- Indexing with the thread's position within the block only.
- No bounds check.
- No waiting for the GPU before checking the results.
- An in-place flip where every thread copies its element to the mirror position. This is a race and produces a mirrored (palindrome-like) array instead of a flipped one.
- Testing only with N = 9, which fits in one block and hides the indexing bug.

Fixes before presenting

- Title slide: the e-mail domain `kueluven.be` should be `kuleuven.be`, and the academic year "2023/24" needs updating.
- Slide 7: `(nvcc))` has an extra closing parenthesis.
- Slides 9 and 10: "seesions" should be "sessions".

- Slide 6: the figure predates Ada, Hopper and Blackwell, and its Pascal and Volta rows are unsupported in CUDA 13.
- Slide 18: “built-in functions” are built-in variables.

Writing the array-flip program, step by step

1. Think before writing anything

1. **Define the mapping.** Element i of the input ends up at position N−1−i of the output.
   - With N = 9, element 0 goes to position 8, element 1 to position 7, and the middle element (4) stays at 4.
   - Every output element depends on exactly one input element, so the problem is embarrassingly parallel: one thread per element.
2. **Choose between out-of-place and in-place.**
   - **Out-of-place** (separate input and output arrays). Thread i reads input element i and writes output element N−1−i. No two threads touch the same location, so there is no race. This is the recommended version for the exercise.
   - **In-place** (one array). If every thread copies its element to the mirror position, one thread can overwrite a value before its partner has read it. That is a race, and the result depends on timing.
   - **The correct in-place version** uses only N/2 threads. Each thread swaps one pair (positions i and N−1−i). For odd N, the middle element simply stays where it is.

2. The kernel (the GPU part)

3. Use a `.cu` source file, or make sure the build passes the `-x cu` option.
4. Write one kernel function:
   - marked as a kernel (`__global__`);
   - returning nothing;
   - taking three parameters: a pointer to the input, a pointer to the output, and N.
5. **Compute the global index:** block index × block size + thread index within the block.
6. **Bounds check:** if the index is N or larger, the thread does nothing.
7. **Do the work:** copy input element *index* to output position N−1−*index*.

3. The main program (the CPU part)

8. **Allocate memory that the GPU can reach.** The simplest option is Unified Memory (`cudaMallocManaged`) for both the input and the output array. The alternative: allocate on the GPU with `cudaMalloc`, keep normal arrays on the CPU, and copy between them with `cudaMemcpy`.
9. **Fill the input on the CPU** with 0, 1, 2, …, N−1. This makes the check trivial: the output must read N−1, N−2, …, 0.
10. **Choose the launch configuration.** Use a block size of 256 threads. The number of blocks is N divided by 256, rounded up.
11. **Launch the kernel** with that number of blocks and threads per block, passing the two pointers and N.
12. **Check that the launch succeeded** (`cudaGetLastError`). An invalid configuration is only reported here.
13. **Wait for the GPU to finish** (`cudaDeviceSynchronize`) and check its return value too. With explicit copies, copying the output back to the CPU also waits for the kernel.
14. **Optional timing.** Record a CUDA event before and after the launch, wait for the second event, and read the elapsed time between the two.
15. **Verify on the CPU.** For every i, the output at position i must equal N−1−i. Report the first mismatch, if any.

16. **Show the result.** Print small arrays to the terminal; write large arrays to a file.
17. **Free the memory** (`cudaFree`).

4. Build, run and test

18. Build with the VS Code task (which calls `nvcc`), then run the program.
19. **Test several sizes:**
    - N = 9, the slide's example;
    - an N that is not a multiple of the block size, such as 1000;
    - an N well above 1024, such as one million, so that many blocks are needed.

    For the in-place version, test both an odd and an even N.
20. **Run the program under `compute-sanitizer`.** It reports out-of-bounds memory accesses that would otherwise go unnoticed.
21. **Break it on purpose,** one change at a time, and note what happens. This is useful material for the report.
    - Remove the bounds check.
    - Remove the wait before verifying.
    - Index with the thread's position within the block only.
    - Run the racy in-place version on a large array.

5. Report

22. Write the single-page report as described for slide 9: the problem, the launch configuration and indexing with justification, how the result was verified, timing if measured, and what the deliberate mistakes showed.